SyrENA

Instruction Set Architecture

1.0

October 11, 2026

Contents

1 Preface **1**

2 Glossary **1**

3 Design **1**
- 3.1 Registers . . . 1
- 3.2 Frame registers . . . 2

4 Register set **2**
- 4.1 Pointer registers . . . 2
  - 4.1.1 rfp - frame pointer . . . 2
  - 4.1.2 rip - instruction pointer . . . 2
  - 4.1.3 rtp - procedure tail pointer . . . 2
  - 4.1.4 rsp - stack pointer . . . 2
- 4.2 Restricted purpose . . . 2
  - 4.2.1 rz - zero register . . . 2
- 4.3 General purpose . . . 2
  - 4.3.1 r0 . . . 2
  - 4.3.2 r1 . . . 3
  - 4.3.3 r2 . . . 3
  - 4.3.4 r3 . . . 3
  - 4.3.5 r4 . . . 3
  - 4.3.6 r5 . . . 3
  - 4.3.7 r6 . . . 3
  - 4.3.8 r7 . . . 3
  - 4.3.9 r8 . . . 3
  - 4.3.10 r9 . . . 3
  - 4.3.11 r10 . . . 3
  - 4.3.12 r11 . . . 3
  - 4.3.13 r12 . . . 3
  - 4.3.14 r13 . . . 3
  - 4.3.15 r14 . . . 4
  - 4.3.16 r15 . . . 4

5 Structures **4**
5.1 CPUID 4

1 Preface

2 Glossary

- Word - 32-bit value
- Half-word - 16-bit value
- Byte - 8-bit value

3 Design

3.1 Registers

The SyrENA architecture, similarily to the TMS9900, does not store general-purpose registers on the CPU, but rather a *frame pointer* that points to the address in register RAM of the current *register frame* for the executing context.

A *register frame* refers to a (virtual) structure in RAM of 128 bytes (1024 bits) in length that stores 32 4-byte values total:

Table 1: Register frame (1023-768)

1023..992	991..960	959..928	927..896	895..864	863..832	831..800	799..768
reserved							

Table 2: Register frame (767-512)

767..736	735..704	703..672	671..640	639..608	607..576	575..544	543..512
reserved					rsp	rtp	rip

Table 3: Register frame (511-256)

511..480	479..448	447..416	415..384	383..352	351..320	319.288	287..256
r15	r14	r13	r12	r11	r10	r9	r8

Table 4: Register frame (255-0)

255..224	223..192	191..160	159..128	127..96	95..64	63..32	31..0
r7	r6	r5	r4	r3	r2	r1	r0

Registers are categorised into two categories:

- **Global** registers are stored directly on the CPU and their values persist between frame changes
- **Local** registers are stored in the *register frame* and their address and/or value depends on the current *frame pointer*

3.2 Frame registers

4 Register set

4.1 Pointer registers

4.1.1 rfp - frame pointer

Availability: Global
Stores the address to the current *register frame*

4.1.2 rip - instruction pointer

Availability: Local
Stores the address of the currently executing instruction

4.1.3 rtp - procedure tail pointer

Availability: Local
Stores the address of the instruction that called the current procedure

4.1.4 rsp - stack pointer

Availability: Local
Stores the address of the current stack frame

4.2 Restricted purpose

4.2.1 rz - zero register

Availability: Global
Always 0. Writes are ignored

4.3 General purpose

4.3.1 r0

Availability: Local

4.3.2 r1

Availability: Local

4.3.3 r2

Availability: Local

4.3.4 r3

Availability: Local

4.3.5 r4

Availability: Local

4.3.6 r5

Availability: Local

4.3.7 r6

Availability: Local

4.3.8 r7

Availability: Local

4.3.9 r8

Availability: Local

4.3.10 r9

Availability: Local

4.3.11 r10

Availability: Local

4.3.12 r11

Availability: Local

4.3.13 r12

Availability: Local

4.3.14 r13

Availability: Local

4.3.15 r14

Availability: Local

4.3.16 r15

Availability: Local

5 Structures

5.1 CPUID

